



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ \_\_\_\_\_ «Информатика и системы управления»

КАФЕДРА \_\_\_\_\_ «Теоретическая информатика и компьютерные технологии»

## Лабораторная работа №1

«Статическая модель сгущения сетки высот»  
по курсу «Моделирование»

Студент: Аригин М.А.

Группа: ИУ9-82Б

Преподаватель: Домрачева А.Б.

Москва 2026

# 1 Постановка задачи

Рассматривается статическая задача о представлении поверхности в фиксированный момент времени  $t_0$ .

Дана аналитическая функция:

$$z = \sin\left(\sqrt{x^2 + y^2}\right),$$

определённая на области  $-3 \leq x \leq 3$ ,  $-3 \leq y \leq 3$ , с диапазоном значений  $-1 \leq z \leq 1$ .

Функция табулируется на регулярной сетке  $n \times n = 10 \times 10$  узлов. Координаты узлов:

$$x_i = x_{\min} + i \cdot \frac{x_{\max} - x_{\min}}{n - 1}, \quad y_j = y_{\min} + j \cdot \frac{y_{\max} - y_{\min}}{n - 1}, \quad i, j = 0, 1, \dots, n - 1.$$

Значения высот в узлах:  $z_{ij} = \sin\left(\sqrt{x_i^2 + y_j^2}\right)$ .

Необходимо вычислить значения функции в  $k \times k = (n - 1) \times (n - 1) = 9 \times 9 = 81$  точках — серединах ячеек сетки:

$$x'_i = \frac{x_i + x_{i+1}}{2}, \quad y'_j = \frac{y_j + y_{j+1}}{2}, \quad i, j = 0, 1, \dots, n - 2.$$

Цель работы — построение статической математической модели поверхности, формализация процесса интерполяции с использованием триангуляции Делоне, постановка и решение вычислительных задач для вычисления значений в серединах ячеек, а также оценка средней абсолютной погрешности интерполяции (с учётом погрешности в узловых точках).

Необходимое условие для корректной постановки задачи: область определения сетки должна быть ограниченной и выпуклой, а значения высот в узлах сетки должны быть заданы для всех узлов.

## 2 Построение математической модели

Построение математической модели представляет собой формализацию процесса описания поверхности на языке математических формул.

Рассматривается поверхность в фиксированный момент времени  $t_0$  при следующих допущениях:

- поверхность описывается функцией  $z = f(x, y) = \sin(\sqrt{x^2 + y^2})$ ;
- функция  $f(x, y)$  непрерывна и дифференцируема в области определения;
- значения высот в узлах исходной сетки известны точно (вычислены аналитически);
- сетка регулярная: шаги  $\Delta x = \frac{x_{\max} - x_{\min}}{n-1}$  и  $\Delta y = \frac{y_{\max} - y_{\min}}{n-1}$  постоянны (метод также применим к псевдoreгулярным сеткам с неравными интервалами).

Введём переменные состояния: координаты точки  $(x, y)$  на плоскости и соответствующая высота  $z = f(x, y)$ .

## 2.1 Модель интерполяции поверхности на основе триангуляции Делоне

Пусть задана сетка с  $n \times n$  узлами  $(x_i, y_j)$ , где  $i = 0, 1, \dots, n-1$ ,  $j = 0, 1, \dots, n-1$ . Для каждого узла  $(x_i, y_j)$  известно значение высоты  $z_{ij} = f(x_i, y_j)$ .

В качестве метода интерполяции поверхности используется триангуляция Делоне [1, 2].

### Определение триангуляции Делоне:

Триангуляцией Делоне называется такая триангуляция набора точек, при которой описанная окружность для каждого треугольника не содержит внутри себя ни одной точки набора [1]. Эта триангуляция хорошо сбалансирована в том смысле, что формируемые треугольники в предельном случае стремятся к равносторонним.

### Алгоритм построения триангуляции Делоне:

Существует несколько подходов к построению триангуляции Делоне [2]. Один из классических — рекурсивный алгоритм с классификацией рёбер на «спящие», «живые» и «мёртвые». Рёбра классифицируются следующим образом:

- «Спящее» ребро — ещё не обнаружено алгоритмом.

- **«Живое» ребро** — обнаружено, известна только одна примыкающая область.
- **«Мёртвое» ребро** — обнаружено и известны обе примыкающие области.

В данной работе для программной реализации используется инкрементальный алгоритм Боуэра—Уотсона [1], который добавляет точки по одной, пересоздавая затронутые треугольники на каждом шаге:

#### **Шаг 1: Инициализация.**

Создаётся супер-треугольник, покрывающий все точки набора. Его вершины выбираются достаточно далеко от области данных.

#### **Шаг 2: Инкрементальное добавление точек.**

Для каждой новой точки  $P$ :

1. Определяются все треугольники, для которых точка  $P$  попадает внутрь описанной окружности («плохие» треугольники). Это треугольники, нарушающие условие Делоне.
2. Формируется полигональная граница «дырки» — множество рёбер, принадлежащих ровно одному «плохому» треугольнику.
3. «Плохие» треугольники удаляются из триангуляции.
4. От каждого граничного ребра к новой точке  $P$  строится новый треугольник.

#### **Шаг 3: Завершение.**

Удаляются все треугольники, содержащие вершины супер-треугольника. Оставшиеся треугольники образуют триангуляцию Делоне исходного набора точек.

#### **Интерполяция высот на основе триангуляции:**

Для произвольной точки  $(x, y)$  значение высоты определяется следующим образом:

1. Находится треугольник триангуляции Делоне, содержащий точку  $(x, y)$ .
2. Используется барицентрическая интерполяция внутри треугольника.

Пусть треугольник образован тремя вершинами  $P_1 = (x_1, y_1, z_1)$ ,  $P_2 = (x_2, y_2, z_2)$ ,  $P_3 = (x_3, y_3, z_3)$ . Барицентрические координаты точки  $(x, y)$  относительно этого треугольника вычисляются по формулам:

$$\begin{aligned}\lambda_1 &= \frac{(y_2 - y_3)(x - x_3) + (x_3 - x_2)(y - y_3)}{(y_2 - y_3)(x_1 - x_3) + (x_3 - x_2)(y_1 - y_3)}, \\ \lambda_2 &= \frac{(y_3 - y_1)(x - x_3) + (x_1 - x_3)(y - y_3)}{(y_2 - y_3)(x_1 - x_3) + (x_3 - x_2)(y_1 - y_3)}, \\ \lambda_3 &= 1 - \lambda_1 - \lambda_2.\end{aligned}$$

Интерполированное значение высоты:

$$z(x, y) = \lambda_1 z_1 + \lambda_2 z_2 + \lambda_3 z_3.$$

### Свойства триангуляции Делоне:

- Интерполяция точно проходит через узлы сетки:  $\hat{f}(x_i, y_j) = z_{ij}$ .
- Функция непрерывна на границах треугольников.
- Внутри каждого треугольника функция является линейной (плоскость), то есть это **линейный алгоритм интерполяции** внутри треугольника.
- Триангуляция Делоне обеспечивает оптимальную сбалансированность треугольников, минимизируя «худшие» треугольники (слишком вытянутые или слишком узкие) [2].
- Погрешность интерполяции для гладких функций составляет  $O(h^2)$ , где  $h$  — характерный размер треугольника [1].
- Метод устойчив к псевдорегулярности сетки: неравные промежутки и неоднозначность соответствия  $x$  и  $y$  не влияют на корректность построения триангуляции.

### Применимость к различным типам сеток:

Триангуляция Делоне применима как к регулярным, так и к псевдорегулярным сеткам [1]. В случае псевдорегулярной сетки координаты отсортированы по возрастанию, но интервалы между узлами не равны. Триангуляция Делоне

строится по всем точкам  $(x_i, y_j)$  независимо от их расположения, что позволяет корректно обрабатывать нерегулярные распределения точек.

В данной работе используется регулярная сетка с равномерным шагом, однако метод без изменений применим и к сеткам с произвольным расположением узлов.

## 3 Решение вычислительных задач

После формализации математической модели необходимо поставить и решить вычислительные задачи для вычисления значений функции в серединах ячеек сетки.

### 3.1 Постановка задачи интерполяции

Задача интерполяции формулируется следующим образом: для заданной сетки  $n \times n = 10 \times 10$  узлов с координатами  $(x_i, y_j)$  и значениями  $z_{ij} = \sin\left(\sqrt{x_i^2 + y_j^2}\right)$  необходимо построить функцию  $\hat{f}(x, y)$ , такую что:

$$\hat{f}(x_i, y_j) = z_{ij}, \quad \forall i, j = 0, 1, \dots, n-1,$$

и вычислить значения  $\hat{f}(x'_i, y'_j)$  в  $k \times k = 9 \times 9 = 81$  серединах ячеек.

Условие единственности решения: для триангуляции Делоне значение  $z(x, y)$  внутри каждого треугольника определяется единственным образом по значениям в трёх вершинах треугольника через барицентрическую интерполяцию.

### 3.2 Вычисление координат середин ячеек

Сетка  $n \times n = 10 \times 10$  узлов образует  $(n-1) \times (n-1) = 9 \times 9$  ячеек. Координаты середин ячеек:

$$x'_i = \frac{x_i + x_{i+1}}{2}, \quad y'_j = \frac{y_j + y_{j+1}}{2}, \quad i, j = 0, 1, \dots, n-2.$$

Для регулярной сетки на  $[-3, 3] \times [-3, 3]$  с шагом  $\Delta = \frac{6}{9} = \frac{2}{3} \approx 0.6\bar{6}$ :

$$x_i = -3 + i \cdot \frac{2}{3}, \quad y_j = -3 + j \cdot \frac{2}{3}, \quad i, j = 0, 1, \dots, 9.$$

Координаты середин:

$$x'_i = -3 + \left(i + \frac{1}{2}\right) \cdot \frac{2}{3}, \quad y'_j = -3 + \left(j + \frac{1}{2}\right) \cdot \frac{2}{3}, \quad i, j = 0, 1, \dots, 8.$$

### 3.3 Алгоритм вычисления значений в серединах ячеек

**Шаг 1: Табулирование функции в узлах сетки.**

Вычисляются значения функции  $z_{ij} = \sin\left(\sqrt{x_i^2 + y_j^2}\right)$  во всех  $n^2 = 100$  узлах сетки.

**Шаг 2: Построение триангуляции Делоне.**

По всем  $n^2 = 100$  точкам  $(x_i, y_j)$  с известными значениями  $z_{ij}$  строится триангуляция Делоне.

**Шаг 3: Интерполяция в серединах ячеек.**

Для каждой из  $k^2 = 81$  точек  $(x'_i, y'_j)$ :

1. Находится треугольник триангуляции Делоне, содержащий точку  $(x'_i, y'_j)$ .
2. Вычисляются барицентрические координаты  $(\lambda_1, \lambda_2, \lambda_3)$  точки относительно этого треугольника.
3. Вычисляется интерполированное значение:

$$\hat{f}(x'_i, y'_j) = \lambda_1 z_1 + \lambda_2 z_2 + \lambda_3 z_3,$$

где  $z_1, z_2, z_3$  — значения функции в вершинах треугольника.

**Шаг 4: Вычисление погрешности.**

Погрешность в каждой точке середины ячейки:

$$e_{ij}^{\text{mid}} = \left| \hat{f}(x'_i, y'_j) - f(x'_i, y'_j) \right| = \left| \hat{f}(x'_i, y'_j) - \sin\left(\sqrt{x'^2_i + y'^2_j}\right) \right|.$$

### 3.4 Оценка погрешности интерполяции

Погрешность интерполяции на основе триангуляции Делоне для функции  $f(x, y)$  с непрерывными вторыми производными оценивается как:

$$|f(x, y) - \hat{f}(x, y)| \leq C \cdot h^2 \cdot \max \left| \frac{\partial^2 f}{\partial x^2} \right|, \quad h = \max(\text{диаметр треугольника}),$$

где  $C$  — константа, зависящая от формы треугольника,  $h$  — максимальный диаметр треугольника в триангуляции.

Для функции  $f(x, y) = \sin(\sqrt{x^2 + y^2})$  вторые производные ограничены на области  $[-3, 3]^2$ , что гарантирует сходимость интерполяции при уменьшении шага сетки.

## 4 Анализ моделей и проверка качества

После построения математической модели и решения вычислительных задач необходимо проверить качество модели, проанализировать полученные результаты и оценить применимость модели.

### 4.1 Анализ решений

Математическая модель триангуляции Делоне позволяет проанализировать различные аспекты интерполяции поверхности  $z = \sin(\sqrt{x^2 + y^2})$ .

**Свойства интерполяции:**

- Интерполяция точно проходит через узлы сетки:  $\hat{f}(x_i, y_j) = z_{ij}$ .
- Функция  $\hat{f}(x, y)$  непрерывна на границах треугольников.
- Внутри каждого треугольника интерполяция линейная (плоскость).
- Триангуляция Делоне обеспечивает оптимальное распределение треугольников, минимизируя «вырожденные» треугольники.

## 4.2 Проверка качества модели

Для проверки качества модели вычислены погрешности интерполяции, сравнивая интерполированные значения с точными значениями аналитической функции.

### 1. Погрешность в узловых точках.

В узловых точках ( $n \times n = 10 \times 10 = 100$  точек) значения табулированы аналитически, поэтому погрешность равна нулю:

$$e_{ij}^{\text{node}} = |z_{ij} - f(x_i, y_j)| = 0, \quad \forall i, j.$$

Средняя абсолютная погрешность в узлах:

$$\text{MAE}_{\text{node}} = \frac{1}{n^2} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} e_{ij}^{\text{node}} = 0.$$

*Замечание:* нулевая погрешность в узлах объясняется тем, что значения вычислены аналитически. В реальных задачах здесь могла бы быть погрешность измерений.

### 2. Погрешность в серединах ячеек.

В серединах ячеек ( $k \times k = 9 \times 9 = 81$  точка) вычислены следующие метрики погрешности:

| Метрика                              | Значение   |
|--------------------------------------|------------|
| Средняя абсолютная погрешность (MAE) | 0.05084292 |
| Среднеквадратичная погрешность (RMS) | 0.07206105 |

Таблица 1: Погрешность интерполяции в серединах ячеек

Метрики вычислены по формулам:

$$\text{MAE}_{\text{mid}} = \frac{1}{k^2} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} e_{ij}^{\text{mid}}, \quad \text{RMS}_{\text{mid}} = \sqrt{\frac{1}{k^2} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} (e_{ij}^{\text{mid}})^2}.$$

### 3. Общая средняя абсолютная погрешность.

Общая средняя абсолютная погрешность с учётом всех точек (узловых и середин ячеек):

$$\begin{aligned} \text{MAE}_{\text{all}} &= \frac{N_{\text{node}} \cdot \text{MAE}_{\text{node}} + N_{\text{mid}} \cdot \text{MAE}_{\text{mid}}}{N_{\text{node}} + N_{\text{mid}}} = \\ &= \frac{100 \cdot 0 + 81 \cdot 0.05084292}{100 + 81} = \frac{4.11828}{181} \approx 0.02275. \end{aligned}$$

| Тип точек                       | Количество | MAE           |
|---------------------------------|------------|---------------|
| Узловые точки ( $n \times n$ )  | 100        | 0.0000        |
| Середины ячеек ( $k \times k$ ) | 81         | 0.0508        |
| <b>Всего</b>                    | <b>181</b> | <b>0.0228</b> |

Таблица 2: Сводная таблица погрешностей

### Качественный анализ погрешности:

- Средняя погрешность 0.051 при диапазоне значений  $[-1, 1]$  составляет около 2.5% от размаха, что является приемлемым результатом для линейной интерполяции на сетке  $10 \times 10$ .
- Погрешность  $O(h^2)$  подтверждается: при шаге  $h = 6/9 \approx 0.667$  ожидаемая погрешность пропорциональна  $h^2 \approx 0.44$ , что согласуется с порядком полученных значений.

Ограничения модели: триангуляция Делоне обеспечивает только непрерывность функции, но не её производных (поверхность кусочно-линейная). Для повышения точности можно увеличить количество узлов сетки или использовать интерполяцию более высокого порядка (например, кубическую на триангуляции).

## 5 Практическая реализация

Listing 1: Модуль интерполяции на основе триангуляции Делоне

```

1 import numpy as np
2
3

```

```

4 class GridRefinement:
5     def __init__(self, X=None, Y=None, Z=None):
6         if X is None:
7             X = []
8         if Y is None:
9             Y = []
10        if Z is None:
11            Z = np.array([[0]])
12
13        self.X = np.array(X, dtype=float)
14        self.Y = np.array(Y, dtype=float)
15        self.Z = np.array(Z, dtype=float)
16
17        if len(self.X) > 0 and len(self.Y) > 0:
18            if self.Z.shape != (len(self.X), len(self.Y)):
19                raise ValueError(
20                    f"Размерность Z должна быть ({len(self.X)}, {len(self.Y)}), "
21                    f"получено {self.Z.shape}"
22                )
23
24        self.N_x = max(0, len(self.X) - 1)
25        self.N_y = max(0, len(self.Y) - 1)
26
27        self.points = None
28        self.values = None
29        self.triangles = None
30
31    def _build_triangulation(self):
32        if self.triangles is not None:
33            return
34
35        if len(self.X) == 0 or len(self.Y) == 0:
36            return
37
38        pts = []
39        vals = []
40        for i in range(len(self.X)):
41            for j in range(len(self.Y)):
42                pts.append([self.X[i], self.Y[j]])
43                vals.append(self.Z[i, j])
44
45        self.points = np.array(pts, dtype=float)
46        self.values = np.array(vals, dtype=float)
47
48        self.triangles = self._bowyer_watson(self.points)
49
50    @staticmethod

```

```

51 def _circumcircle(p1, p2, p3):
52     ax, ay = p1
53     bx, by = p2
54     cx, cy = p3
55
56     D = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
57
58     if abs(D) < 1e-12:
59         mid_x = (ax + bx + cx) / 3.0
60         mid_y = (ay + by + cy) / 3.0
61         return mid_x, mid_y, float('inf')
62
63     a_sq = ax * ax + ay * ay
64     b_sq = bx * bx + by * by
65     c_sq = cx * cx + cy * cy
66
67     ux = (a_sq * (by - cy) + b_sq * (cy - ay) + c_sq * (ay - by)) / D
68     uy = (a_sq * (cx - bx) + b_sq * (ax - cx) + c_sq * (bx - ax)) / D
69
70     r_sq = (ax - ux) ** 2 + (ay - uy) ** 2
71     return ux, uy, r_sq
72
73 @staticmethod
74 def _bowyer_watson(points):
75     n = len(points)
76     if n < 3:
77         return []
78
79     x_min = points[:, 0].min()
80     x_max = points[:, 0].max()
81     y_min = points[:, 1].min()
82     y_max = points[:, 1].max()
83
84     dx = x_max - x_min
85     dy = y_max - y_min
86     d_max = max(dx, dy, 1e-6)
87
88     mid_x = (x_min + x_max) / 2.0
89     mid_y = (y_min + y_max) / 2.0
90
91     p0 = [mid_x - 20.0 * d_max, mid_y - d_max]
92     p1 = [mid_x + 20.0 * d_max, mid_y - d_max]
93     p2 = [mid_x, mid_y + 20.0 * d_max]
94
95     all_points = list(points) + [p0, p1, p2]
96     all_points_arr = np.array(all_points, dtype=float)
97

```

```

98     triangles = {(n, n + 1, n + 2)}
99
100     cc_cache = {}
101     t0 = (n, n + 1, n + 2)
102     cc_cache[t0] = GridRefinement._circumcircle(
103         all_points_arr[t0[0]], all_points_arr[t0[1]], all_points_arr[t0[2]]
104     )
105
106     for idx in range(n):
107         px, py = all_points_arr[idx]
108
109         bad_triangles = set()
110         for tri in triangles:
111             cx, cy, r_sq = cc_cache[tri]
112             dist_sq = (px - cx) ** 2 + (py - cy) ** 2
113             if dist_sq < r_sq + 1e-10:
114                 bad_triangles.add(tri)
115
116         edge_count = {}
117         for tri in bad_triangles:
118             edges = [
119                 (tri[0], tri[1]),
120                 (tri[1], tri[2]),
121                 (tri[0], tri[2]),
122             ]
123             for e in edges:
124                 e_sorted = tuple(sorted(e))
125                 edge_count[e_sorted] = edge_count.get(e_sorted, 0) + 1
126
127         boundary_edges = [e for e, cnt in edge_count.items() if cnt == 1]
128
129         for tri in bad_triangles:
130             triangles.discard(tri)
131             cc_cache.pop(tri, None)
132
133         for e in boundary_edges:
134             new_tri = tuple(sorted((e[0], e[1], idx)))
135             triangles.add(new_tri)
136             cc_cache[new_tri] = GridRefinement._circumcircle(
137                 all_points_arr[new_tri[0]],
138                 all_points_arr[new_tri[1]],
139                 all_points_arr[new_tri[2]],
140             )
141
142     super_indices = {n, n + 1, n + 2}
143     result = []
144     for tri in triangles:

```

```

145         if super_indices.isdisjoint(tri):
146             result.append(tri)
147
148     return result
149
150     @staticmethod
151     def _barycentric_coords(px, py, x1, y1, x2, y2, x3, y3):
152         denom = (y2 - y3) * (x1 - x3) + (x3 - x2) * (y1 - y3)
153
154         if abs(denom) < 1e-14:
155             return -1.0, -1.0, -1.0
156
157         lam1 = ((y2 - y3) * (px - x3) + (x3 - x2) * (py - y3)) / denom
158         lam2 = ((y3 - y1) * (px - x3) + (x1 - x3) * (py - y3)) / denom
159         lam3 = 1.0 - lam1 - lam2
160
161         return lam1, lam2, lam3
162
163     def delaunay_interpolation(self, x, y):
164         if self.triangles is None:
165             self._build_triangulation()
166
167         if self.triangles is None or len(self.triangles) == 0:
168             return self.Z[0, 0] if self.Z.size > 0 else 0.0
169
170         eps = -1e-10
171
172         for tri in self.triangles:
173             i0, i1, i2 = tri
174             x1, y1 = self.points[i0]
175             x2, y2 = self.points[i1]
176             x3, y3 = self.points[i2]
177
178             lam1, lam2, lam3 = self._barycentric_coords(
179                 x, y, x1, y1, x2, y2, x3, y3
180             )
181
182             if lam1 >= eps and lam2 >= eps and lam3 >= eps:
183                 z = (lam1 * self.values[i0]
184                     + lam2 * self.values[i1]
185                     + lam3 * self.values[i2])
186                 return z
187
188         dists = (self.points[:, 0] - x) ** 2 + (self.points[:, 1] - y) ** 2
189         nearest = np.argmin(dists)
190         return self.values[nearest]

```

## Listing 2: Вычислительный скрипт с визуализацией

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib import cm
4 from mpl_toolkits.mplot3d import Axes3D
5 from grid_refinement import GridRefinement
6
7 plt.rcParams['figure.figsize'] = (16, 8)
8 plt.rcParams['font.size'] = 10
9
10 def analytical_function(x, y):
11     r = np.sqrt(x**2 + y**2)
12     return np.sin(r)
13
14 n = 10
15 k = n - 1
16 x_min, x_max = -3.0, 3.0
17 y_min, y_max = -3.0, 3.0
18
19 x_nodes = np.linspace(x_min, x_max, n)
20 y_nodes = np.linspace(y_min, y_max, n)
21 X_grid, Y_grid = np.meshgrid(x_nodes, y_nodes, indexing='ij')
22 Z_nodes = analytical_function(X_grid, Y_grid)
23
24 fig = plt.figure(figsize=(16, 6))
25 ax1 = fig.add_subplot(121, projection='3d')
26 surf = ax1.plot_surface(X_grid, Y_grid, Z_nodes, cmap=cm.viridis,
27                         alpha=0.9, linewidth=0.3, antialiased=True)
28 ax1.set_xlabel('X')
29 ax1.set_ylabel('Y')
30 ax1.set_zlabel('Z')
31 ax1.set_title(f'Исходная сетка {n}x{n} узлов')
32 plt.colorbar(surf, ax=ax1, shrink=0.6, aspect=20)
33 ax2 = fig.add_subplot(122)
34 contour = ax2.contourf(X_grid, Y_grid, Z_nodes, levels=20, cmap=cm.viridis)
35 ax2.set_xlabel('X')
36 ax2.set_ylabel('Y')
37 ax2.set_title('Контурная карта исходной сетки')
38 ax2.set_aspect('equal')
39 plt.colorbar(contour, ax=ax2, shrink=0.8)
40 plt.tight_layout()
41 plt.show()
42
43 grid = GridRefinement(x_nodes.copy(), y_nodes.copy(), Z_nodes.copy())
44
45 x_midpoints = [(x_nodes[i] + x_nodes[i+1]) / 2.0 for i in range(k)]
```

```

46 y_midpoints = [(y_nodes[j] + y_nodes[j+1]) / 2.0 for j in range(k)]
47 x_midpoints = np.array(x_midpoints)
48 y_midpoints = np.array(y_midpoints)
49 X_mid_grid, Y_mid_grid = np.meshgrid(x_midpoints, y_midpoints, indexing='ij')
50
51 Z_midpoints_interp = np.zeros((k, k))
52 for i in range(k):
53     for j in range(k):
54         Z_midpoints_interp[i, j] = grid.delaunay_interpolation(
55             X_mid_grid[i, j], Y_mid_grid[i, j])
56
57 Z_midpoints_exact = analytical_function(X_mid_grid, Y_mid_grid)
58
59 Z_nodes_exact = analytical_function(X_grid, Y_grid)
60 error_nodes = np.abs(Z_nodes - Z_nodes_exact)
61 mean_error_nodes = np.mean(error_nodes)
62 max_error_nodes = np.max(error_nodes)
63
64 error_midpoints = np.abs(Z_midpoints_interp - Z_midpoints_exact)
65 mean_error_midpoints = np.mean(error_midpoints)
66 max_error_midpoints = np.max(error_midpoints)
67 rms_error_midpoints = np.sqrt(np.mean(error_midpoints**2))
68
69 total_nodes = n * n
70 total_midpoints = k * k
71 total_points = total_nodes + total_midpoints
72 total_mean_error = (mean_error_nodes * total_nodes +
73                     mean_error_midpoints * total_midpoints) / total_points
74
75 fig = plt.figure(figsize=(18, 12))
76 ax1 = fig.add_subplot(2, 3, 1)
77 im1 = ax1.contourf(X_mid_grid, Y_mid_grid, Z_midpoints_interp,
78                   levels=20, cmap=cm.viridis)
79 ax1.set_xlabel('X')
80 ax1.set_ylabel('Y')
81 ax1.set_title(f'Интерполированные значения ({k}x{k})')
82 ax1.set_aspect('equal')
83 plt.colorbar(im1, ax=ax1, shrink=0.8)
84 ax2 = fig.add_subplot(2, 3, 2)
85 im2 = ax2.contourf(X_mid_grid, Y_mid_grid, Z_midpoints_exact,
86                   levels=20, cmap=cm.viridis)
87 ax2.set_xlabel('X')
88 ax2.set_ylabel('Y')
89 ax2.set_title(f'Точные значения ({k}x{k})')
90 ax2.set_aspect('equal')
91 plt.colorbar(im2, ax=ax2, shrink=0.8)
92 ax3 = fig.add_subplot(2, 3, 3)

```

```

93 im3 = ax3.contourf(X_mid_grid, Y_mid_grid, error_midpoints,
94                   levels=20, cmap=cm.Reds)
95 ax3.set_xlabel('X')
96 ax3.set_ylabel('Y')
97 ax3.set_title(f'Погрешность (макс: {max_error_midpoints:.6f})')
98 ax3.set_aspect('equal')
99 plt.colorbar(im3, ax=ax3, shrink=0.8)
100 ax4 = fig.add_subplot(2, 3, 4, projection='3d')
101 ax4.plot_surface(X_mid_grid, Y_mid_grid, Z_midpoints_interp,
102                cmap=cm.viridis, alpha=0.9, linewidth=0.3)
103 ax4.set_xlabel('X')
104 ax4.set_ylabel('Y')
105 ax4.set_zlabel('Z')
106 ax4.set_title('3D: Интерполированные')
107 ax5 = fig.add_subplot(2, 3, 5, projection='3d')
108 ax5.plot_surface(X_mid_grid, Y_mid_grid, Z_midpoints_exact,
109                cmap=cm.viridis, alpha=0.9, linewidth=0.3)
110 ax5.set_xlabel('X')
111 ax5.set_ylabel('Y')
112 ax5.set_zlabel('Z')
113 ax5.set_title('3D: Точные')
114 ax6 = fig.add_subplot(2, 3, 6)
115 diff = Z_midpoints_interp - Z_midpoints_exact
116 im6 = ax6.contourf(X_mid_grid, Y_mid_grid, diff,
117                   levels=20, cmap=cm.RdYlBu_r)
118 ax6.set_xlabel('X')
119 ax6.set_ylabel('Y')
120 ax6.set_title(f'Разница (средняя: {np.mean(diff):.6f})')
121 ax6.set_aspect('equal')
122 plt.colorbar(im6, ax=ax6, shrink=0.8)
123 plt.tight_layout()
124 plt.show()
125
126 print("=" * 60)
127 print("СТАТИСТИКА ПОГРЕШНОСТЕЙ")
128 print("=" * 60)
129 print(f"\nУзловые точки ({n}x{n} = {total_nodes}):")
130 print(f"  MAE: {mean_error_nodes:.10f}")
131 print(f"  MAX: {max_error_nodes:.10f}")
132 print(f"\nСредины ячеек ({k}x{k} = {total_midpoints}):")
133 print(f"  MAE: {mean_error_midpoints:.8f}")
134 print(f"  MAX: {max_error_midpoints:.8f}")
135 print(f"  RMS: {rms_error_midpoints:.8f}")
136 print(f"\n{' ' * 60}")
137 print(f"ОБЩАЯ MAE ({total_points} точек): {total_mean_error:.8f}")
138 print(f"{' ' * 60}")
139

```

## 6 Результаты

Погрешность в узловых точках ( $10 \times 10 = 100$  точек):

Средняя абсолютная: 0.0000000000  
Максимальная: 0.0000000000  
RMS: 0.0000000000

Погрешность в серединах ячеек ( $9 \times 9 = 81$  точек):

Средняя абсолютная: 0.05084292  
Максимальная: 0.45413806  
RMS: 0.07206105

ОБЩАЯ СРЕДНЯЯ АБСОЛЮТНАЯ ПОГРЕШНОСТЬ (с учётом всех 181 точек):  
0.02275291

В результате построения математической модели и решения вычислительных задач получены следующие результаты:

- **Табулирование функции:** функция  $z = \sin(\sqrt{x^2 + y^2})$  табулирована на регулярной сетке  $10 \times 10$  узлов в области  $[-3, 3]^2$ . Диапазон значений в узлах: от  $-0.892$  до  $0.992$ .
- **Интерполяция в серединах ячеек:** с помощью триангуляции Делоне вычислены значения в  $9 \times 9 = 81$  серединах ячеек. Диапазон интерполированных значений: от  $-0.612$  до  $0.990$ .
- **Погрешность интерполяции:** средняя абсолютная погрешность в серединах ячеек составила  $MAE_{\text{mid}} = 0.051$ .
- **Общая погрешность:** с учётом нулевой погрешности в 100 узловых точках и погрешности в 81 середине ячейки, общая средняя абсолютная погрешность составила  $MAE_{\text{all}} \approx 0.023$ .
- **Относительная точность:** при диапазоне значений функции  $[-1, 1]$  средняя погрешность 0.051 составляет около 2.5% от размаха, что является приемлемым результатом для линейной интерполяции.

## 7 Выводы

- Выполнена формализация задачи интерполяции поверхности  $z = \sin\left(\sqrt{x^2 + y^2}\right)$  на языке математических формул. Поверхность описана через триангуляцию Делоне [1] с барицентрической интерполяцией [2], что является основой статической математической модели.
- Поставлены и решены вычислительные задачи: табулирование функции на сетке  $10 \times 10$  узлов, построение триангуляции Делоне, вычисление интерполированных значений в  $9 \times 9 = 81$  серединах ячеек, оценка погрешности интерполяции.
- Проведена проверка качества модели. Средняя абсолютная погрешность интерполяции в серединах ячеек:  $MAE_{\text{mid}} = 0.051$ . Общая средняя абсолютная погрешность (с учётом узловых точек):  $MAE_{\text{all}} = 0.023$ .
- Погрешность в узловых точках равна нулю, так как значения табулированы аналитически. В реальных задачах здесь могла бы быть погрешность измерений.
- Метод триангуляции Делоне устойчив и применим как к регулярным, так и к псевдoreгулярным сеткам. Ограничения модели: интерполяция обеспечивает только непрерывность функции, но не её производных (поверхность кусочно-линейная).

## Список литературы

- [1] Скворцов А. В. Триангуляция Делоне и её применение. — Томск: Изд-во Том. ун-та, 2002. — 128 с.
- [2] Де Берг М., Ван Кревельд М., Овермарс М., Шварцкопф О. Вычислительная геометрия: алгоритмы и приложения / Пер. с англ. — М.: ДМК Пресс, 2004. — 352 с.